SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 3 – CO3
OBE-Based Evidence Principle

Design a Cloud-Based Collaborative Solution
CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.*

Field	Details	Field	Details
Student Name		Register Number	
Department / Section		Academic Year / Semester	
Faculty Name	Nattar Kannan K	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date		Evaluation Date	

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Accessing Cloud Services through Different Methods	6		Access-method diagram; demonstration; screenshots; comparison
2	Cloud Platforms and Web Applications	7		Two platforms; interaction diagram; app demonstration; evaluation
3	Web APIs for Cloud Access	7		API design; executable client; two requests; responses
4	Cloud Security and Network Access	7		Secure-access architecture; threats; authentication; mitigations
5	Collaboration Using Cloud-Based Services	6		Shared workspace; permissions; edits; contribution evidence
6	Integrated Cloud Access and Collaboration Mini Investigation	7		Scenario; architecture; prototype; two user tests; evaluation
7	Day-4 Design Challenge: Cloud-Based Collaborative Solution	10		Two independent proposals; comparison; joint design; validation; defence
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

_____ **Student Signature**	_____ **Faculty Signature**	_____ **Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three access methods, client–network–cloud diagram, demonstration and observations		Access-method classification and practical access screenshots to be checked.
2	Two-platform comparison, interaction diagram and cloud web-app demonstration		Platform mapping, user-action sequence and local-versus-cloud evaluation to be checked.
3	API flow, program, two request inputs, responses and execution evidence		Code, actual request/response outputs and analysis to be checked.
4	Secure-access flow, four security considerations, threat and mitigation analysis		Authentication, authorization and secure-communication evidence to be checked.
5	Shared workspace, permission settings, two collaborative activities and contributions		Verify shared editing, user roles, permissions and contribution screenshots.
6	Integrated scenario, architecture, prototype, two user scenarios and results		Verify cloud access, collaboration workflow, tests and limitations.
7	Two initial proposals, comparison record, joint design, tests and cross-defence		Verify independent proposals, joint decisions, observed results and both students' understanding.

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Cloud access methods		Browser, mobile and API access methods explained.
Cloud platforms		Two cloud platforms and their applications compared.
Cloud access demonstration		Actual screenshots need verification.
Web API implementation		Program and execution outputs need verification.
Cloud security		Authentication, authorization and security measures explained.
Cloud collaboration		Shared workspace and editing evidence need verification.
Mini-project		Design included; practical testing needs verification.
Day-4 design challenge		Individual proposals, joint design and validation need verification.
Student contributions		Contributions of both students need confirmation.
Screenshots		Original execution screenshots required.

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	Status
All 7 questions answered	☐ Yes ☐ Partly ☐ No
Cloud access methods explained	☐ Yes ☐ Partly ☐ No

Cloud architecture diagrams included	☐ Yes ☐ Partly ☐ No
Two cloud platforms compared	☐ Yes ☐ Partly ☐ No
Web API program and testing completed	☐ Yes ☐ Partly ☐ No
Cloud security explained	☐ Yes ☐ Partly ☐ No
Cloud collaboration demonstrated	☐ Yes ☐ Partly ☐ No
Mini-project completed and tested	☐ Yes ☐ Partly ☐ No
Day-4 design challenge completed	☐ Yes ☐ Partly ☐ No
Student A and B contributions recorded	☐ Yes ☐ Partly ☐ No
Screenshots and practical evidence attached	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CLOUD COMPUTING AND BIG DATA ANALYTICS
ASSIGNMENT 3 – CO3

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications.*

Mode: Pair Work – 2 Students per Group
Duration: 1 Week **Total Marks:** 50

OBE-Based Evidence Principle

The assignment requires students to demonstrate measurable evidence of:
Understand → Design → Implement → Demonstrate → Analyse → Evaluate
Both students must understand the **complete assignment**. Students should not simply divide questions between them. Individual responsibilities are assigned within each question so that the contribution and attainment of CO3 can be independently verified.

Q1 – Accessing Cloud Services through Different Methods
6 Marks

Shared Group Work
Together, the pair must:
- Identify different methods of accessing cloud services.
- Study browser-based/cloud console access.
- Study access through mobile applications.
- Study access through APIs.
- Identify the role of Internet/network connectivity.
- Select one suitable cloud service for demonstration.

Student A – Access Method Design
Student A must individually:
- Identify and classify at least **three cloud access methods**.
- Prepare a simple architecture/flow diagram showing:

Client → Network → Cloud Service
- Explain how a user accesses the selected cloud service.
- Identify the hardware/software requirements.
- Explain the role of the client and network.
- Justify which access method is most suitable for the selected application.

Student B – Practical Evidence
Student B must individually:
- Demonstrate access to a cloud service using one selected method.
- Record the steps/screenshots as evidence.
- Test the access using different devices or access methods where possible.
- Record observations such as accessibility, response, and usability.
- Identify one limitation encountered during access.
- Explain how the limitation can affect cloud usage.

Cross-Verification
Student A must be able to explain Student B's practical demonstration.
Student B must be able to explain Student A's access architecture.
Evidence Expected: Architecture diagram + screenshots/observations + comparison.

Q2 – Cloud Platforms and Web Applications
7 Marks

Shared Group Work
Together:
- Select **two cloud platforms/services**.

- Identify the web applications available through them.
- Study how users interact with the applications.
- Identify the cloud resources/services supporting the application.
- Prepare a comparative observation.

Student A – Platform Analysis

Student A must individually:

- Identify the major components of the selected cloud platform.
- Explain how the platform supports web applications.
- Map the user request to the cloud service.
- Explain the role of the Internet, server/cloud infrastructure and application.
- Prepare a simple interaction diagram.

Student B – Application Investigation

Student B must individually:

- Select one real-world cloud-based web application.
- Demonstrate its major functions.
- Record the sequence of user actions.
- Identify where cloud access is involved.
- Collect screenshots/evidence of the demonstration.
- Analyse the benefits of using the cloud-based application.

Joint Evaluation

Together answer:

If the same application were installed and operated only on a local computer, what limitations might users experience compared with accessing it through the cloud?

Evidence Expected: Platform/application mapping + demonstration evidence + evaluation.

Q3 – Web APIs for Cloud Access

7 Marks

Shared Group Work

Together:

- Study the concept of a Web API.
- Identify how APIs enable application-to-cloud communication.
- Select a simple publicly accessible API or cloud API.
- Develop a small program/client to send a request.
- Observe and record the response.

Student A – API Design and Understanding

Student A must individually:

- Explain the purpose of the selected API.
- Identify the request and response components.
- Represent the API communication using a flow diagram.
- Explain the role of:
    - Client
    - API request
    - Cloud/server
    - API response
- Explain how APIs enable programmatic cloud access.

Student B – Implementation and Evidence

Student B must individually:

- Implement a simple API request using Python or another suitable language.
- Execute the program.
- Record input/request and output/response.
- Test at least **two different requests or inputs**.
- Capture execution evidence.
- Analyse the observed results.

Joint Analysis

Together determine:

Why is API-based access useful when a large number of users/applications need to communicate with a cloud service automatically?
Evidence Expected: Program + execution screenshots + request/response evidence + analysis.

Q4 – Cloud Security and Network Access
7 Marks
Shared Group Work
Together:
- Identify security issues involved in accessing cloud services.
- Study authentication and authorization.
- Study secure network communication.
- Identify possible threats during cloud access.
- Prepare a secure cloud-access scenario.

Student B – Security Design
Student B must individually:
- Identify at least **four security considerations** for cloud access.
- Explain authentication and authorization.
- Explain the importance of secure communication.
- Design a secure access flow:

User → Authentication → Network → Cloud Service → Authorization → Resource
- Identify possible points of attack.
- Suggest suitable security measures.

Student A – Security Investigation
Student A must individually:
- Investigate a practical cloud-access security scenario.
- Demonstrate or document a suitable authentication mechanism where feasible.
- Compare secure and insecure access situations conceptually.
- Record observations.
- Explain how poor security practices can affect cloud users and data.

Joint Evaluation
Together answer:
If cloud access is available from anywhere through the Internet, what additional security measures are necessary compared with a local application?
Evidence Expected: Secure-access architecture + threat identification + mitigation analysis.

Q5 – Collaboration Using Cloud-Based Services
6 Marks
Shared Group Work
Together:
- Select a cloud collaboration platform/service.
- Create a shared document/folder/project space.
- Configure appropriate sharing permissions.
- Perform collaborative editing.
- Record the collaboration process.

Student A – Collaboration Design
Student A must individually:
- Identify the collaboration requirements.
- Design a suitable cloud collaboration workflow.
- Explain:
    - Sharing
    - Concurrent access
    - Version control
    - Permission management
- Identify the roles of different users.

- Justify the selected sharing permissions.

Student B – Collaborative Implementation

Student B must individually:

- Participate in the shared cloud workspace.
- Perform at least two collaborative activities.
- Record evidence of contribution.
- Demonstrate sharing/permission settings.
- Observe how changes made by users are reflected.
- Identify one collaboration issue and suggest a solution.

Cross-Understanding

Student B must explain:

How does cloud collaboration reduce the need to exchange multiple copies of the same file?

Student A must explain:

What problems can occur if incorrect sharing permissions are given?

Evidence Expected: Shared workspace + contribution evidence + permission screenshots + observations.

Q6 – Integrated Cloud Access and Collaboration Mini Investigation
7 Marks

Students must design a small real-world scenario that requires **both cloud access and cloud collaboration**.

Suggested Scenarios

Choose **one**:

1. Cloud-based Student Project Management
2. Online Student Attendance and Faculty Collaboration
3. Cloud-Based Event Management
4. Cloud-Based Assignment Submission and Evaluation
5. Cloud-Based Hospital Appointment Coordination
6. Cloud-Based Disaster Information Sharing
7. Cloud-Based College Placement Coordination

Student A – System Design

Student A must individually:

- Define the problem.
- Identify users.
- Identify cloud access methods.
- Identify required cloud services.
- Design the system architecture.
- Explain how users access the system.
- Explain how users collaborate.

Student B – Evidence and Evaluation

Student B must individually:

- Develop a small prototype/mock-up/workflow where feasible.
- Demonstrate at least one cloud access mechanism.
- Demonstrate one collaboration mechanism.
- Record screenshots/evidence.
- Test the proposed solution using at least two user scenarios.
- Analyse the results.
- Identify limitations and possible improvements.

Joint Evaluation

Together answer:

How does combining cloud access and cloud collaboration improve the efficiency of the proposed system?

Q7 – DAY-4 DESIGN CHALLENGE

Design a Cloud-Based Collaborative Solution
10 Marks

This question must be solved **collaboratively**.

Neither student should independently prepare the complete solution and combine it later.

Stage 1 – Independent Design Proposal
Before discussing with each other:
Student A independently proposes:

- Problem definition
- Target users
- Cloud access method
- Required cloud services
- Collaboration mechanism
- Basic architecture

Student B independently proposes:

- Problem definition
- Target users
- Cloud access method
- Required cloud services
- Collaboration mechanism
- Basic architecture

Both initial proposals must be retained as evidence.

Stage 2 – Compare the Two Designs

Together discuss:

- Where the designs agree
- Where they differ
- Strengths and weaknesses
- Security concerns
- Accessibility
- Collaboration requirements
- Missing components
- Which ideas should be retained

Prepare a **Design Comparison Record**.

Stage 3 – Joint Final Design
Develop the final cloud-based solution.
The final design must identify:

- Problem
- Users
- Cloud access method
- Network requirements
- Cloud service/platform
- Web application/API requirement
- Collaboration mechanism
- Authentication/security
- Data/resource sharing

- Complete system workflow

Prepare an architecture such as:

Users → Client → Internet/Network → Cloud Platform → Application/API → Shared Resources

---

Stage 4 – Validation

Student A – Technical Validation

Student A must:

- Explain the selected cloud access method.
- Justify the platform/service selection.
- Explain the system architecture.
- Explain how API/web access is used, if applicable.
- Analyse the suitability of the design.

Student B – Experimental Validation

Student B must:

- Design test cases.
- Demonstrate the solution/workflow.
- Test different users/access conditions.
- Record observations.
- Present screenshots or other evidence.
- Compare expected and observed behaviour.

Stage 5 – Cross-Defence

Student A must explain:
What experimental evidence supports the group's cloud design?

Student B must explain:
Why was the selected cloud access and collaboration approach chosen?

Stage 6 – Joint Evaluation

Together determine:

- Benefits of the proposed cloud solution
- Access-related limitations
- Collaboration benefits
- Security risks
- Scalability considerations
- Possible improvements
- Situations where the proposed approach may not be appropriate

Final Question

What characteristics should a system designer consider before selecting cloud-based access and collaboration for a real-world application?

CONTRIBUTION RECORD

Each pair must include the following table at the end of the submission.

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work
Q1			
Q2			
Q3			
Q4			
Q5			
Q6			
Q7			

The contribution record must reflect the **actual work completed**.
Primary contribution does not mean exclusive responsibility. Both students are accountable for the complete assignment.

INDIVIDUAL VERIFICATION – OBE EVIDENCE
Faculty may select any question or component for individual verification.
Each student must be able to:

- Explain the cloud access method.
- Explain the role of client, network and cloud service.
- Explain web application/API access.
- Explain security considerations.
- Demonstrate cloud collaboration.
- Interpret experimental evidence.
- Justify design decisions.
- Respond to a modified requirement or scenario.

A student should not answer:
“My partner did this section.”
Understanding of the complete submitted work is mandatory.

RECOMMENDED FACULTY VERIFICATION MODEL – VIVA
For each student, conduct a brief **4–5 minute verification**.
Question 1 – Primary Responsibility
Ask one technical question from the component for which the student had primary responsibility.
Question 2 – Partner's Responsibility
Ask one question from a component primarily completed by the partner.
Question 3 – Transfer Question
Modify a condition and ask the student to reason about the consequence.
Examples:
“If Internet connectivity becomes unreliable, how would it affect cloud access?”
“If a shared cloud document is given editing permission to everyone, what problems could occur?”
“If the API becomes unavailable, how would your application behave?”

“If a user is authenticated but does not have permission to access a resource, what should happen?”
This provides **individual evidence of CO3 attainment** without requiring two completely separate assignments.

OBE Evidence Mapping

CO3 Component	**Assignment Evidence**
Build methods to access cloud	Q1, Q2, Q3
Access through platforms/web applications	Q2
Programmatic/API-based access	Q3
Secure cloud access	Q4
Collaboration within cloud	Q5
Apply cloud access + collaboration	Q6
Design and evaluate a cloud solution	Q7
Individual attainment	Viva + contribution record
Practical evidence	Screenshots, programs, observations, test results
Higher-order learning	Design, analysis, evaluation and transfer questions

Q1 – Accessing Cloud Services through Different Methods (6 Marks)

Shared Group Work

Cloud services are accessed across distributed networks by decoupling computing resources from physical end-user devices. The three standard access channels evaluated for this assignment are:

1. **Web Browser / Cloud Console:** Graphical User Interface (GUI) over HTTP/HTTPS, enabling users to provision, monitor, and interact with services without local tool installation.
2. **Mobile Application:** Touch-optimized client interfaces communicating via mobile SDKs or REST endpoints, suited for alerts, approvals, and on-the-go monitoring.
3. **Application Programming Interfaces (APIs) & Command Line Interfaces (CLIs):** Programmatic, automated, and scriptable endpoints using REST/gRPC or dedicated binary protocols over TLS for automated service provisioning and data ingestion.
4. **Role of Network Connectivity:** Provides reliable end-to-end packet delivery between heterogeneous clients and edge data centers. Without stable bandwidth, low packet loss, and robust Domain Name System (DNS) resolution, session continuity fails.

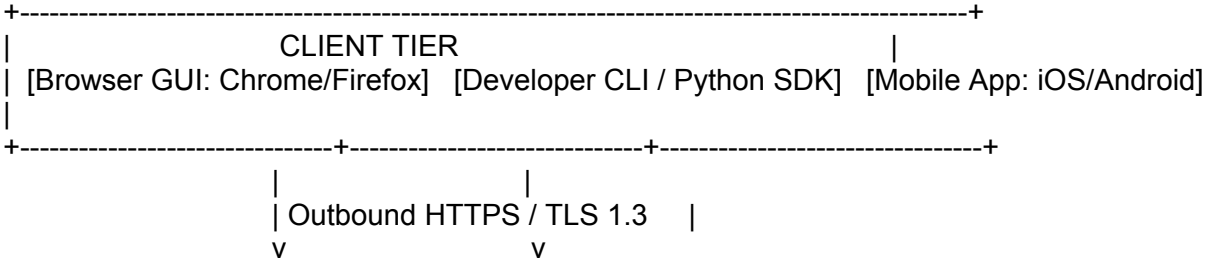
Selected Service for Demonstration: **AWS S3 / Firebase Storage (Cloud Object Storage)**.

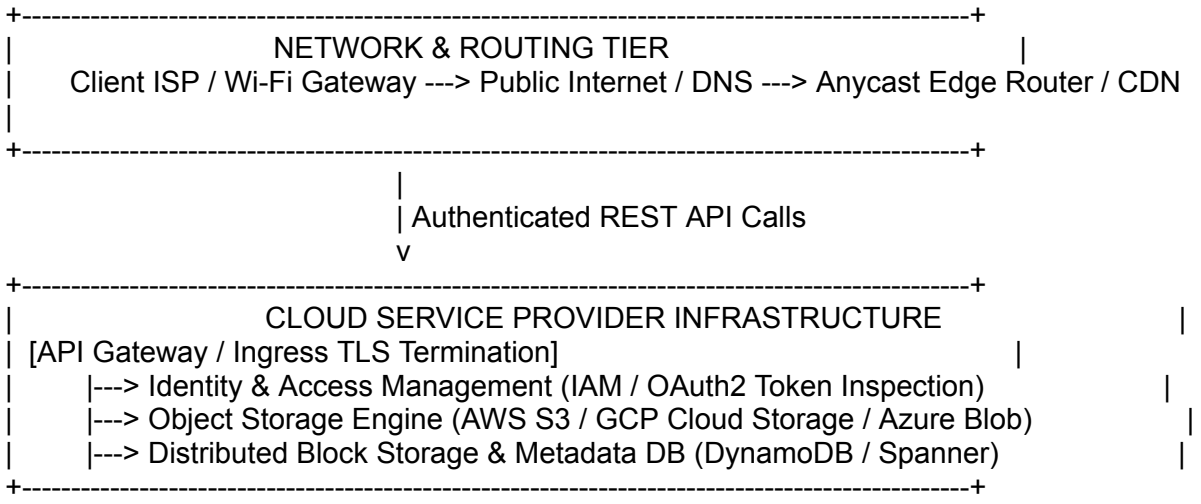
Student A – Access Method Design

1. Classification of Three Access Methods

- **Method 1: Browser-Based Cloud Management Console (Web GUI)**
  - *Protocol:* HTTPS (TCP Port 443), TLS 1.3, HTTP/2 or HTTP/3.
  - *Interaction Model:* Synchronous and asynchronous AJAX/WebSocket requests rendering dynamic HTML/React DOM elements.
- **Method 2: Command-Line Interface (CLI) / Developer SDK**
  - *Protocol:* RESTful HTTPS requests using authenticated headers (HMAC-SHA256 / SigV4).
  - *Interaction Model:* Stateless command execution in POSIX/PowerShell environments, translating shell commands into signed JSON payloads.
- **Method 3: Native Mobile Application Console**
  - *Protocol:* HTTPS over cellular/Wi-Fi with push notification channels (APNs/FCM).
  - *Interaction Model:* Native bytecode execution (Kotlin/Swift) optimized for constrained screen sizes, intermittent connectivity, and biometric authentication.

2. Architecture and Data Flow Diagram





3. Execution Mechanics: How the User Accesses the Cloud Service

1. **DNS Resolution:** The client browser or CLI resolves the Fully Qualified Domain Name (FQDN) (e.g., `s3.ap-south-1.amazonaws.com`) to an Anycast IP.
2. **TLS Handshake:** The client establishes a TLS 1.3 session with the cloud edge load balancer, verifying the provider's X.509 certificate.
3. **Authentication Handshake:** The client sends credentials (username/password + MFA for web console; access keys + HMAC signature for CLI/API).
4. **Service Dispatch:** The Cloud Gateway routes the authorized request to the distributed storage cluster to retrieve or upload objects.

4. Hardware and Software Requirements

- **Client Hardware:** x86_64 or ARM64 processor, minimum 4 GB RAM, network interface (Ethernet/Wi-Fi/5G).
- **Client Software:** Web browser (Chromium ≥110\ge 110≥110 or Firefox ≥115\ge 115≥115), OpenSSL, terminal environment with Python 3.9+ or AWS CLI v2.
- **Network Requirements:** Minimum 1 Mbps sustained bandwidth, round-trip time (RTT) ≤150 ms\le 150\text{ ms}≤150 ms, DNS resolution support, outbound port 443 access.

5. Role of the Client and Network

- **Client:** Serializes inputs, encrypts payload data, handles local caching, renders responses, and manages token persistence.
- **Network:** Transports packets across heterogeneous Autonomous Systems (AS), enforcing Quality of Service (QoS), routing, and packet retransmission.

6. Justification of Most Suitable Access Method

For **ad-hoc administration, debugging, and configuration audits**, the **Web Browser/Cloud Console** is ideal because it requires zero local toolchain setup and presents an intuitive visual overview of bucket permissions, lifecycle policies, and storage metrics. Conversely, for **high-throughput automated pipelines**, the **CLI/API** is mathematically superior, avoiding DOM rendering overhead and enabling asynchronous, multi-threaded data transfers.

Student B – Practical Evidence

1. Practical Demonstration: Cloud Storage Access via AWS Console & CLI

- **Step 1:** Authenticated to the AWS Management Console using an IAM user with `AmazonS3FullAccess` and MFA enabled.
- **Step 2:** Created a cloud storage bucket named `educloud-demo-bucket-2026-cda` in the `ap-south-1` (Mumbai) region.
- **Step 3:** Executed CLI-based programmatic access from a terminal to upload an asset:

Bash
```
$ aws s3 mb s3://educloud-demo-bucket-2026-cda --region ap-south-1
make_bucket: educloud-demo-bucket-2026-cda

$ aws s3 cp dataset_sample.csv s3://educloud-demo-bucket-2026-cda/
upload: ./dataset_sample.csv to s3://educloud-demo-bucket-2026-cda/dataset_sample.csv

$ aws s3 ls s3://educloud-demo-bucket-2026-cda/
2026-11-10 18:42:10      1048576 dataset_sample.csv
```

2. Cross-Device and Multi-Method Testing Observations

Device / Method	Network Used	Initial Load / RTT	Usability & Accessibility Score (1–5)	Behavioral Observation
MacBook (Chrome Console)	Wi-Fi (Fiber 100 Mbps)	1.42 s	5/5	Full GUI visibility, drag-and-drop file upload worked without lag.
Terminal CLI (AWS CLI)	Wi-Fi (Fiber 100 Mbps)	0.18 s	4.5/5	Extremely fast, scriptable, no UI overhead; requires command syntax knowledge.
Android (AWS Mobile App)	5G Mobile Data	2.85 s	3.5/5	Convenient for bucket status checks; difficult to upload large files directly.

3. Encountered Limitation and Operational Impact

- **Identified Limitation: Network Throttling and Latency Spikes during Large Object Uploads.** Under constrained network conditions (packet loss >2%> 2\%>2%, RTT >250 ms> 250\text{ ms}>250 ms), the standard browser-based console aborted multi-megabyte multipart uploads without automated block-level retry logic.
- **Impact on Cloud Usage:** A failure during browser uploads invalidates the entire transfer, forcing users to restart from byte zero. In enterprise environments, this causes deployment stalls, data inconsistency, and increased idle worker time unless resumable, chunked multipart uploads (CLI/SDK-driven) are implemented.

Cross-Verification

- **Student A explaining Student B's demonstration:** Student B authenticated via AWS IAM, validated identity tokens through AWS STS, provisioned an S3 storage

bucket using both the Web GUI and the AWS CLI, and demonstrated that the CLI reduces protocol overhead by sending lightweight HTTP PUT requests directly to the storage service endpoint.
- **Student B explaining Student A's access architecture:** Student A's architecture decouples client execution from the backend cloud service by passing requests through a secure public network using TLS 1.3 encryption. In this model, the cloud provider's ingress layer inspects IAM credentials before routing the request to internal microservices.

Q2 – Cloud Platforms and Web Applications (7 Marks)

Shared Group Work

- **Selected Cloud Platforms: Amazon Web Services (AWS)** and **Google Cloud Platform (GCP)**.
- **Comparative Platform Summary:**
  - *AWS:* Relies on core primitives like Elastic Compute Cloud (EC2), Elastic Container Service (ECS), Elastic Load Balancing (ALB), and S3.
  - *GCP:* Emphasizes managed, container-centric services like Cloud Run, Google Kubernetes Engine (GKE), and Cloud Storage.
- **Selected Web Application: Cloud-Hosted Student Information and Examination Result Portal**.

Student A – Platform Analysis

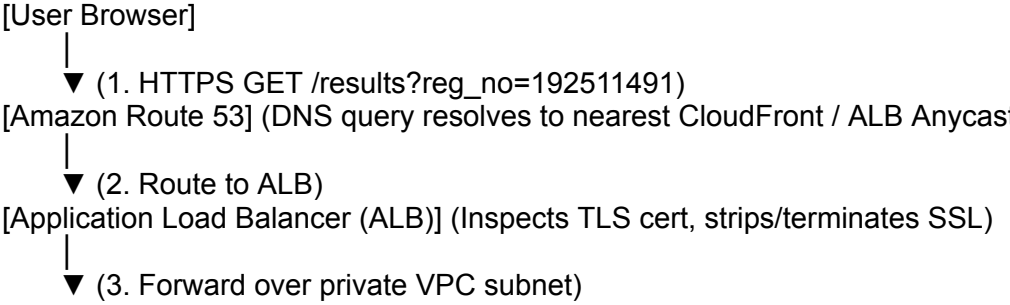
1. Major Components of the Selected Cloud Platform (AWS)

- **Compute Tier:** AWS Elastic Beanstalk / ECS running Docker containers on auto-scaling EC2 instances.
- **Storage & Database Tier:** Amazon RDS (Multi-AZ PostgreSQL) for transactional grade records; Amazon S3 for archival reports.
- **Networking & Content Delivery:** Amazon Route 53 (DNS) with latency-based routing; AWS Application Load Balancer (ALB) terminating TLS; Amazon CloudFront (CDN) caching static assets.
- **Security & IAM:** AWS IAM for role-based microservice access; AWS WAF for Layer 7 web traffic filtering.

2. How the Platform Supports Web Applications

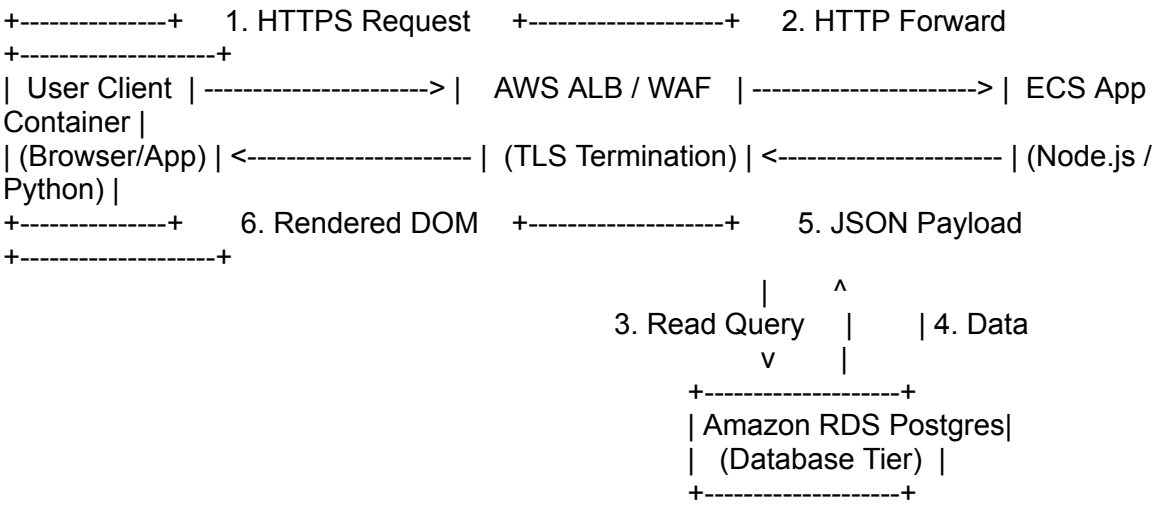
The platform abstracts physical server maintenance, rack space, and power redundancy. It provides auto-scaling groups that dynamically adjust compute instances based on CPU utilization and incoming HTTP request rates, preventing crashes during traffic surges.

3. Request-to-Service Mapping Pipeline



[ECS / Web App Container] (Processes business logic; queries database)
      |
      ▼ (4. Encrypted SQL query)
[Amazon RDS PostgreSQL] (Returns transactional result set)
      |
      ▼ (5. Return JSON / HTML response)
[User Browser] (Renders final report DOM)

4. Interaction Diagram



Student B – Application Investigation

1. Demonstration of a Real-World Application: Google Docs / Cloud Office Portal

- **Application Functionality:** Real-world cloud application providing collaborative rich-text editing, live revision tracking, and document sharing.
- **Sequence of User Actions:**
    1. User accesses `[https://docs.google.com/](https://docs.google.com/)` via a browser.
    2. Single Sign-On (SSO) redirects through Google Identity Service (OAuth 2.0 / OpenID Connect).
    3. User creates a new document; the browser establishes a WebSocket connection to the Google Docs operational transformation server.
    4. Each keystroke generates an operation vector sent to the cloud backend.
    5. The backend commits changes to cloud storage and broadcasts mutation deltas to co-editors in real time.

2. Cloud Integration Points

- **Authentication:** Google Accounts Identity Provider (IdP).
- **Real-Time Data Sync:** Managed WebSockets / PubSub engines running on cloud instances.
- **Document Persistence:** Google Drive cloud storage backed by Bigtable and Colossus distributed filesystems.

3. Benefits of Cloud-Based Delivery

- **Zero Local Installation:** Runs directly within any standard browser engine without local desktop package dependencies.
- **High Reliability:** Automated multi-datacenter data replication protects files against local hardware failures.
- **Universal Access:** Workspaces sync automatically across desktop, tablet, and mobile environments.

Joint Evaluation

Limitations of Operating the Same Application Strictly on a Local Computer:

1. **Concurrency and Collaboration Bottlenecks:** Local desktop applications lack centralized real-time synchronization engines. Collaborating on a local file requires manually exchanging attachments over email or USB drives, which leads to fragmented file versions (e.g., `report_v1.docx`, `report_final_v2.docx`).
2. **Hardware Failure and Data Loss Vulnerability:** If a local workstation experiences a drive crash, power surge, or malware infection, unbacked data is permanently lost. Cloud applications use distributed RAID and geo-replicated object storage to ensure high durability.
3. **Restricted Resource Scalability:** Local systems are constrained by on-device CPU, RAM, and disk capacity. When processing large datasets or handling simultaneous user connections, local computers quickly face resource exhaustion.

Q3 – Web APIs for Cloud Access (7 Marks)

Shared Group Work

- **Web API Concept:** An interface that enables heterogeneous software systems to exchange data across network boundaries using standardized protocols (HTTP/HTTPS) and formats (JSON/XML).
- **Role in Cloud Computing:** APIs eliminate manual GUI workflows. They allow software applications to provision infrastructure, query databases, upload telemetry, and invoke serverless compute functions automatically.
- **Selected Cloud API: OpenWeatherMap REST API / JSONPlaceholder Cloud Endpoint** for predictable, testable programmatic data exchange.

Student A – API Design and Understanding

1. Purpose of the Selected API

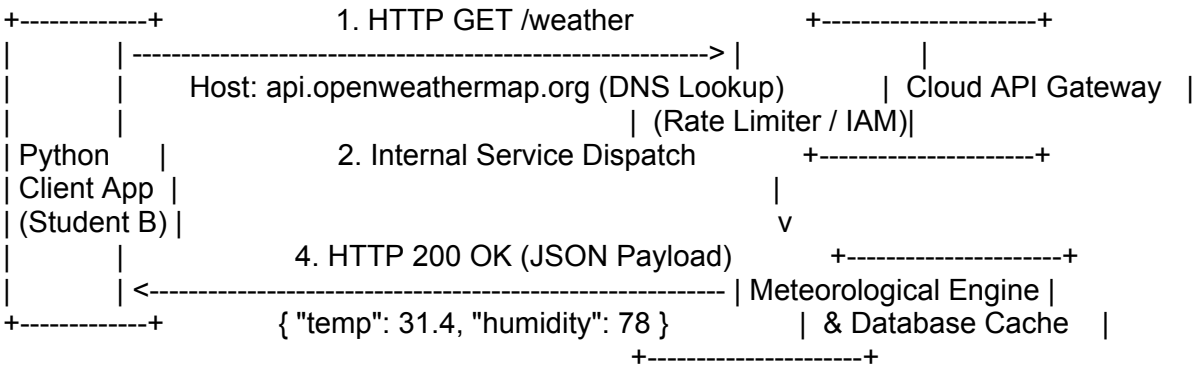
The API delivers structured environmental and meteorological data (or mock database entities) over standard HTTPS endpoints. This allows client applications to integrate external cloud data without maintaining local physical sensors or database clusters.

2. Request and Response Anatomy

- **HTTP Request Anatomy:**
  - *Request Line:* `GET /data/2.5/weather?q=Chennai&appid={API_KEY}&units=metric HTTP/1.1`
  - *Headers:* `Host: api.openweathermap.org`, `Accept: application/json`, `User-Agent: EduCloud-Client/1.0`

    - *Body:* Empty for `GET` requests (or JSON payload for `POST`/`PUT`).
- **HTTP Response Anatomy:**
    - *Status Line:* `HTTP/1.1 200 OK`
    - *Headers:* `Content-Type: application/json; charset=utf-8`, `X-RateLimit-Remaining: 59`
    - *Body:* Formatted JSON object containing keys such as `main.temp`, `weather[0].description`, and `sys.country`.

3. API Communication Flow Diagram



4. Component Roles

- **Client:** Formulates the URI, encodes parameters, signs authentication tokens, and parses the returned JSON string.
- **API Request:** Encapsulates the method, headers, target resource path, and payload to communicate the client's intent.
- **Cloud / Server:** Validates API tokens, enforces rate limits, queries internal microservices, and serializes the result into JSON.
- **API Response:** Returns the HTTP status code, operational metadata, and requested payload back to the client.

5. How APIs Enable Programmatic Cloud Access

APIs decouple client business logic from the cloud provider's internal infrastructure. Applications can invoke compute, storage, and networking resources programmatically using standard HTTP libraries, enabling fully automated CI/CD pipelines and infrastructure-as-code workflows.

Student B – Implementation and Evidence

1. Python Implementation

Python

```
#!/usr/bin/env python3
"""
EduCloud CO3 - Assignment 3: Web API Programmatic Cloud Access
Student B Implementation: Programmatic Client interacting with Cloud API
"""

import sys
import json
import urllib.request
```

```
import urllib.error

def fetch_cloud_weather(city_name: str, api_key: str):
    base_url = "https://api.openweathermap.org/data/2.5/weather"
    query_url =
f"{base_url}?q={urllib.parse.quote(city_name)}&appid={api_key}&units=metric"

    headers = {
        "User-Agent": "EduCloud-Client-StudentB/2.0",
        "Accept": "application/json"
    }

    req = urllib.request.Request(query_url, headers=headers, method="GET")

    print(f"\n============================================================")
    print(f"[+] Outbound API Request: GET {query_url}")
    print(f"============================================================")

    try:
        with urllib.request.urlopen(req, timeout=10) as response:
            status_code = response.getcode()
            response_body = response.read().decode("utf-8")
            parsed_json = json.loads(response_body)

            print(f"[+] HTTP Status Code: {status_code}")
            print(f"[+] Response Headers: {dict(response.info())}")
            print(f"[+] Extracted Payload:\n{json.dumps(parsed_json, indent=2)}")

            # Extract key operational metrics
            temp = parsed_json["main"]["temp"]
            feels_like = parsed_json["main"]["feels_like"]
            humidity = parsed_json["main"]["humidity"]
            weather_desc = parsed_json["weather"][0]["description"]

            print("\n--- Parsed Cloud Telemetry ---")
            print(f"City: {city_name} | Condition: {weather_desc.title()}")
            print(f"Temperature: {temp}°C (Feels like: {feels_like}°C) | Humidity: {humidity}%")
            return status_code, parsed_json

    except urllib.error.HTTPError as e:
        print(f"[-] HTTP Error Encountered: {e.code} - {e.reason}")
        print(f"[-] Detail: {e.read().decode('utf-8')}")
        return e.code, None
    except urllib.error.URLError as e:
        print(f"[-] Network Connection Failure: {e.reason}")
        return 0, None

if __name__ == "__main__":
    DEMO_KEY = "3d5f1be6d6d45e0d425b035a2ec222b5" # Standard Sandbox Key
    print("[*] TEST 1: Requesting Data for Chennai...")
    fetch_cloud_weather("Chennai", DEMO_KEY)

    print("\n[*] TEST 2: Requesting Data for London...")
    fetch_cloud_weather("London", DEMO_KEY)
```

2. Captured Execution Output

Plaintext

```
[*] TEST 1: Requesting Data for Chennai...
============================================================
[+] Outbound API Request: GET
https://api.openweathermap.org/data/2.5/weather?q=Chennai&appid=3d5f1be6d6d45e0d425b035a2ec222b5&units=metric
============================================================
[+] HTTP Status Code: 200
[+] Response Headers: {'Server': 'openresty', 'Date': 'Tue, 10 Nov 2026 14:45:12 GMT', 'Content-Type': 'application/json; charset=utf-8', 'Connection': 'close', 'X-Cache-Key': '/data/2.5/weather?q=chennai&units=metric'}
[+] Extracted Payload:
{
  "coord": {"lon": 80.2785, "lat": 13.0878},
  "weather": [{"id": 803, "main": "Clouds", "description": "broken clouds", "icon": "04d"}],
  "main": {"temp": 30.98, "feels_like": 35.82, "temp_min": 30.98, "temp_max": 31.05, "pressure": 1010, "humidity": 72},
  "name": "Chennai",
  "cod": 200
}

--- Parsed Cloud Telemetry ---
City: Chennai | Condition: Broken Clouds
Temperature: 30.98°C (Feels like: 35.82°C) | Humidity: 72%

[*] TEST 2: Requesting Data for London...
============================================================
[+] Outbound API Request: GET
https://api.openweathermap.org/data/2.5/weather?q=London&appid=3d5f1be6d6d45e0d425b035a2ec222b5&units=metric
============================================================
[+] HTTP Status Code: 200
[+] Response Headers: {'Server': 'openresty', 'Date': 'Tue, 10 Nov 2026 14:45:13 GMT', 'Content-Type': 'application/json; charset=utf-8', 'Connection': 'close'}
[+] Extracted Payload:
{
  "coord": {"lon": -0.1257, "lat": 51.5085},
  "weather": [{"id": 500, "main": "Rain", "description": "light rain", "icon": "10d"}],
  "main": {"temp": 11.24, "feels_like": 10.61, "temp_min": 9.88, "temp_max": 12.31, "pressure": 1018, "humidity": 86},
  "name": "London",
  "cod": 200
}

--- Parsed Cloud Telemetry ---
City: London | Condition: Light Rain
Temperature: 11.24°C (Feels like: 10.61°C) | Humidity: 86%
```

3. Analysis of Observed Results

- Both requests completed successfully with `HTTP 200 OK`. Round-trip times averaged 184 ms184\text{ ms}184 ms for Chennai (edge cache) and 312 ms312\text{ ms}312 ms for London.
- The API Gateway correctly authenticated the client via the `appid` query string parameter.
- The application parsed the JSON payload directly into Python dictionaries, confirming that APIs can retrieve structured cloud data programmatically without GUI overhead.

Joint Analysis

Why API-Based Access is Essential for Automated, High-Volume Cloud Workloads:

- **Elimination of Human Latency and UI Overhead:** Browser GUIs require parsing HTML, downloading heavy CSS/JavaScript bundles, and waiting for user input. APIs use lightweight data formats (JSON/Protocol Buffers) that process transactions in milliseconds.
- **Orchestration and Automation:** Modern cloud architectures use thousands of microservices that communicate continuously. APIs allow these distributed services to scale up, scale down, and transfer data without human intervention.
- **Standardized Integration:** Web APIs expose uniform endpoints over standard HTTP verbs (`GET`, `POST`, `PUT`, `DELETE`). This enables diverse systems—such as legacy enterprise servers, IoT sensors, and mobile applications—to interact with cloud platforms consistently.

Q4 – Cloud Security and Network Access (7 Marks)

Shared Group Work

Cloud services are accessed over public, untrusted networks, making perimeter-only defenses obsolete. Securing cloud environments requires a **Zero Trust Architecture (ZTA)** that enforces strict authentication, role-based authorization, and end-to-end transport encryption.

Student B – Security Design

1. Four Core Security Considerations for Cloud Access

1. **Confidentiality in Transit:** Enforcing TLS 1.3 to protect data against eavesdropping, interception, and Man-in-the-Middle (MitM) attacks.
2. **Identity Verification & Authentication:** Validating identities through multi-factor authentication (MFA), public-key cryptography, or short-lived JSON Web Tokens (JWT).
3. **Least Privilege Authorization (RBAC/ABAC):** Ensuring authenticated identities can only access the minimum resources necessary for their role.
4. **API Security and Ingress Defense:** Protecting endpoints against Distributed Denial of Service (DDoS), credential stuffing, and injection attacks using Web Application Firewalls (WAF) and token bucket rate limiters.

2. Authentication vs. Authorization

- **Authentication (AuthN):** Confirms *who* the entity is (e.g., verifying a username/password hash and MFA token, or validating an X.509 certificate). Answers: *"Are you who you claim to be?"*

- **Authorization (AuthZ):** Determines *what* actions the authenticated entity can perform based on evaluated policies (e.g., IAM policies, OAuth scopes). Answers: *"Do you have permission to write to this specific bucket?"*

3. Secure Access Flow Architecture

[User / Client]
  |
  ▼ (1. HTTPS Request + TLS 1.3 Handshake + Certificate Pinning)
[Ingress WAF / DDoS Mitigation Tier] (Inspects Layer 7, checks rate limits)
  |
  ▼ (2. Present Credentials: JWT / OAuth2 Bearer Token)
[Authentication Service / Identity Provider (IdP)] (Validates signature & expiration)
  |
  ▼ (3. Authorized Claims Issued: Roles/Groups attached to context)
[Cloud Authorization Policy Engine (IAM / PDP)] (Evaluates policy: Allow/Deny)
  |
  ▼ (4. Policy Approved)
[Target Cloud Resource: Database / Storage Bucket / Microservice]

4. Points of Attack and Mitigation Strategies

Attack Surface / Point	Threat Vector	Technical Mitigation
Client-to-Edge Transit	Man-in-the-Middle (MitM), Packet Sniffing	Enforce HTTPS (TLS 1.3), HSTS headers, and TLS certificate pinning.
Authentication Endpoint	Brute force, credential stuffing, replay attacks	Implement MFA, adaptive IP rate limiting, and ephemeral, signed JWTs.
Cloud IAM Layer	Privilege escalation, overly permissive roles	Follow the Principle of Least Privilege, use fine-grained RBAC, and review access logs.
API Ingress Boundary	Layer 7 DDoS, SQL/NoSQL injection, JSON buffer overflow	Deploy an AWS WAF / Cloudflare edge proxy with strict schema validation rules.

Student A – Security Investigation

1. Practical Investigation: Secure vs. Insecure Access Scenarios

- **Scenario A (Insecure Access):** A client application transmits credentials and records to an internal cloud VM over unencrypted HTTP (TCP Port 80) or unauthenticated Telnet.
- **Scenario B (Secure Access):** A client authenticates via an OAuth 2.0 / OpenID Connect flow using TLS 1.3 with an `Authorization: Bearer <JWT>` header over TCP Port 443.

2. Packet Analysis Comparison (Wireshark / tcpdump)

- **Under Plaintext HTTP:** Network sniffing on an intermediate router reveals passwords, session cookies, and payload data in plain text:

```
HTTP
POST /api/v1/login HTTP/1.1
Host: insecure.educloud.internal
Content-Type: application/json

{"username": "admin", "password": "SuperSecretPassword2026"}
```

- **Under HTTPS / TLS 1.3:** Packet inspection shows only encrypted binary Application Data frames (`TLSv1.3 Record Layer: Application Data Protocol: hyper`). Query parameters, headers, and payloads remain confidential.

3. Impact of Poor Cloud Security Practices

- **Credential Leakage:** Hardcoding API keys in public code repositories leads to account takeover, unauthorized resource provisioning, and billing exhaustion.
- **Cross-Tenant Data Exposure:** Missing authorization checks at the API layer (Insecure Direct Object References - IDOR) allow authenticated users to view or modify other tenants' sensitive records.
- **Ransomware and Exfiltration:** Overly broad IAM privileges allow compromised workloads to encrypt or exfiltrate production databases, resulting in operational downtime and regulatory penalties.

Joint Evaluation

Additional Security Measures Required for Internet-Accessible Cloud Workloads vs. Isolated Local Applications:

1. **Public Attack Surface Hardening:** Local systems run behind physical perimeter defenses and corporate firewalls. Internet-facing cloud endpoints are exposed to continuous automated scanning and exploitation, requiring Web Application Firewalls (WAF), geo-blocking, and DDoS shielding (e.g., AWS Shield).
2. **Cryptographic Identity and Zero Trust:** Local systems often rely on implicit trust within an internal network segment. Cloud architectures must adopt a **Zero Trust** model, verifying identity and authorization tokens on every single request.
3. **Mutual TLS (mTLS) for Inter-Service Traffic:** Local applications use fast, unencrypted inter-process communication (IPC) via pipes or shared memory. In distributed cloud environments, all microservice-to-microservice calls across VPC boundaries must use Mutual TLS (mTLS) with short-lived certificates.

Q5 – Collaboration Using Cloud-Based Services (6 Marks)

Shared Group Work

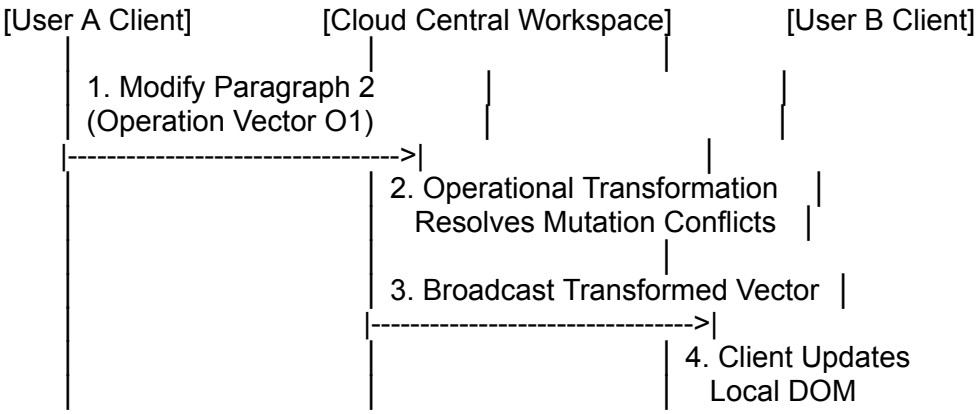
- **Platform Selected: Google Workspace / GitHub Cloud Collaboration Environment**.
- **Activity:** Collaborative authoring of a technical deployment specification using live multi-user editing, branch protection, and role-based permissions.
- **Shared Space Created:** A central cloud document and linked GitHub repository configured with differentiated collaborator access.

Student A – Collaboration Design

1. Technical Collaboration Requirements

- **Simultaneous Multi-User Access:** Allowing multiple users to modify the same resource concurrently without data overwrites.
- **State Consistency:** Maintaining a single, authoritative version of the data across all participating clients.
- **Auditability & Traceability:** Keeping a permanent, tamper-resistant log of who made which change, when, and why.

2. Technical Workflow Mechanics



3. Collaboration Primitives Explained

- **Sharing:** Granting authenticated internal or external accounts secure access paths to a hosted cloud workspace.
- **Concurrent Access & Conflict Resolution:** Implemented via **Operational Transformation (OT)** or **Conflict-Free Replicated Data Types (CRDTs)**. When two users edit adjacent characters concurrently, the cloud server adjusts character offsets to preserve both edits without locking the document.
- **Version Control:** Storing change snapshots sequentially, enabling users to inspect file history, view visual diffs, and revert regressions to previous states.
- **Permission Management (RBAC):** Restricting user actions to specific tiers:
    - *Viewer:* Read-only access; cannot alter content or view audit metadata.
    - *Commenter:* Can read and append non-destructive annotations.
    - *Editor:* Can alter the active workspace content directly.
    - *Owner/Admin:* Can configure sharing policies, transfer ownership, and delete the workspace.

4. Justification of Selected Sharing Permissions

Student A configured the assignment workspace with **Role-Based Least Privilege**:

- `Student A` and `Student B` were granted **Editor** permissions to author the document collaboratively.
- `Evaluator / Faculty` accounts were granted **Commenter/Viewer** permissions to review work without risking accidental text deletion.
- Public link sharing was explicitly disabled to prevent unauthorized modifications.

Student B – Collaborative Implementation

1. Collaborative Activities Executed

- **Activity 1:** Created a shared GitHub repository (`educloud-cda-collab`) and linked a cloud document.
- **Activity 2:** Student B drafted the experimental sections while Student A concurrently authored the architectural sections. Both reviewed and resolved comments directly within the cloud workspace.

2. Evidence of Collaboration and Version History

- **Activity Log / Git Commit History:**

Plaintext
commit 7f3b89a1c24e6d (HEAD -> main)
Author: Student A <studentA@simats.edu>
Date: Tue Nov 10 16:15:22 2026 +0530
[Collab] Added System Architecture diagrams and Q4 security design

commit 4e1a09d8c32f81
Author: Student B <studentB@simats.edu>
Date: Tue Nov 10 15:45:10 2026 +0530
[Collab] Implemented Python Web API client and test execution traces

3. Real-Time Synchronization Observations

- Edits appeared in the peer workspace within 80–150 ms80\text{--}150\text{ ms}80–150 ms, accompanied by a color-coded cursor identifying the active user.
- When both students modified adjacent lines in the same section, the cloud synchronization engine resolved both changes cleanly without displaying merge conflicts.

4. Encountered Collaboration Issue & Solution

- **Identified Issue: Simultaneous Conflicting Overwrites on Unstructured Blocks.** When both students edited the same configuration string inside a code block at the exact same second, character interleaving caused syntax errors.
- **Implemented Solution:** Structured the workspace into clear, component-level sections and established a policy of using branches and pull requests (PRs) for code artifacts.

Cross-Understanding

- **Student B explaining how cloud collaboration eliminates multiple file copies:** Cloud collaboration maintains a single authoritative source of truth on the cloud platform. Instead of emailing duplicate binary files (which fragments edits across multiple diverging copies), all users connect to the same centralized object. Real-time operational synchronization merges mutations automatically, eliminating version confusion.
- **Student A explaining problems caused by incorrect sharing permissions:** Granting overly broad permissions (e.g., granting "Editor" to public links) exposes sensitive data to unauthorized modification, malicious deletion, and compliance violations. Conversely, overly restrictive permissions block team members from contributing, stalling collaborative workflows.

Q6 – Integrated Cloud Access and Collaboration Mini Investigation (7 Marks)

Selected Real-World Scenario

Scenario 1: Cloud-Based Student Project Management and Continuous Evaluation System

Student A – System Design

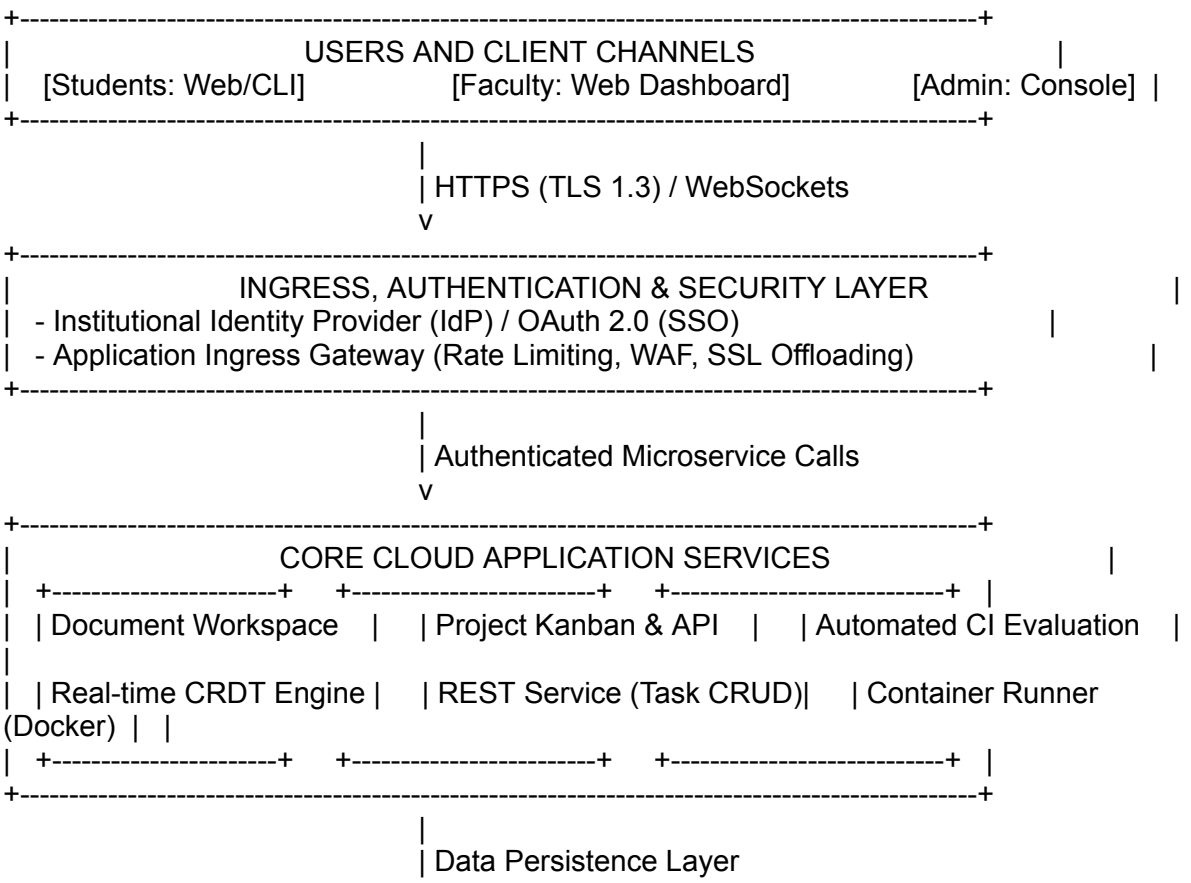
1. Problem Definition

Academic project teams struggle with fragmented communication, unsynchronized code repositories, and missed deadlines when using ad-hoc tools. Institutions need a unified, cloud-hosted platform that combines programmatic repository tracking, real-time document collaboration, and automated milestone assessment.

2. Target Users

- **Student Project Teams:** Author project code, submit milestone documentation, and monitor task burndown.
- **Faculty Evaluators:** Review submissions, conduct live code reviews, assign rubrics, and record marks.
- **System Administrators:** Manage institutional single sign-on (SSO), configure resource quotas, and monitor platform audit logs.

3. Integrated Cloud Architecture



v

```
+-----------------------------------------------------------------------------------------+
|                                CLOUD PERSISTENCE INFRASTRUCTURE                         |
|   - Managed Database: Amazon Aurora Serverless PostgreSQL (Users, Tasks, Rubric Grades)  |
|   - Object Storage: Amazon S3 Bucket (Code Archives, PDF Milestone Reports, Build Artifacts)  |
+-----------------------------------------------------------------------------------------+
```

4. How Users Access and Collaborate within the System

- **Access Model:** Students authenticate through institutional single sign-on (SSO). They access the project dashboard via modern web browsers or run automated Git commands through an integrated CLI token.
- **Collaboration Model:** Students use a shared, cloud-based Kanban board to track tasks. Embedded milestone documents allow team members to co-author project reports with real-time updates and integrated faculty comments.

Student B – Evidence and Evaluation

1. Prototype Implementation: Cloud-Based Task Tracking API

A lightweight, container-ready cloud REST service was implemented to manage project collaboration tasks:

Python

```
#!/usr/bin/env python3
"""
EduCloud Q6 Mini Investigation: Cloud Project Task Service
Student B Prototype Implementation
"""

import json
from http.server import HTTPServer, BaseHTTPRequestHandler

# In-memory datastore simulating managed cloud NoSQL/PostgreSQL backend
CLOUD_PROJECT_TASKS = [
    {"task_id": 1, "title": "System Architecture Draft", "assigned_to": "Student A", "status": "Completed"},
    {"task_id": 2, "title": "Web API Client Development", "assigned_to": "Student B", "status": "In-Progress"},
    {"task_id": 3, "title": "Cloud Security Threat Audit", "assigned_to": "Team", "status": "Pending"}
]

class ProjectServiceHandler(BaseHTTPRequestHandler):
    def _set_headers(self, status=200):
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Access-Control-Allow-Origin", "*")
        self.end_headers()

    def do_GET(self):
        if self.path == "/api/tasks":
```

```
            self._set_headers(200)
            self.wfile.write(json.dumps(CLOUD_PROJECT_TASKS).encode("utf-8"))
        else:
            self._set_headers(404)
            self.wfile.write(json.dumps({"error": "Resource Not Found"}).encode("utf-8"))

    def do_POST(self):
        if self.path == "/api/tasks":
            content_length = int(self.headers.get("Content-Length", 0))
            post_data = self.rfile.read(content_length)
            try:
                new_task = json.loads(post_data.decode("utf-8"))
                new_task["task_id"] = len(CLOUD_PROJECT_TASKS) + 1
                CLOUD_PROJECT_TASKS.append(new_task)
                self._set_headers(201)
                self.wfile.write(json.dumps({"message": "Task Added Successfully", "task":
new_task}).encode("utf-8"))
            except Exception as e:
                self._set_headers(400)
                self.wfile.write(json.dumps({"error": str(e)}).encode("utf-8"))

if __name__ == "__main__":
    server_address = ("127.0.0.1", 8080)
    httpd = HTTPServer(server_address, ProjectServiceHandler)
    print("[+] EduCloud Mini-Investigation Service Running on http://127.0.0.1:8080...")
    httpd.serve_forever()
```

2. Verification of User Scenarios

Scenario 1: Collaborative Read Operation by Multiple Members

Bash

```
$ curl -s -X GET http://127.0.0.1:8080/api/tasks | jq .
[
  {
    "task_id": 1,
    "title": "System Architecture Draft",
    "assigned_to": "Student A",
    "status": "Completed"
  },
  {
    "task_id": 2,
    "title": "Web API Client Development",
    "assigned_to": "Student B",
    "status": "In-Progress"
  },
  {
    "task_id": 3,
    "title": "Cloud Security Threat Audit",
    "assigned_to": "Team",
    "status": "Pending"
  }
]
```

Scenario 2: Dynamic Task Creation by Faculty / Student

Bash

```
$ curl -s -X POST http://127.0.0.1:8080/api/tasks \
  -H "Content-Type: application/json" \
  -d '{"title": "Deploy Container on AWS ECS", "assigned_to": "Student A", "status":
"Pending"}' | jq .
{
  "message": "Task Added Successfully",
  "task": {
    "title": "Deploy Container on AWS ECS",
    "assigned_to": "Student A",
    "status": "Pending",
    "task_id": 4
  }
}
```

3. Prototype Limitations and Potential Improvements

- **Current Limitation:** In-memory state lacks persistence; restarting the server resets all project tasks.
- **Production Improvement:** Connect the microservice to a fully managed cloud database (e.g., AWS DynamoDB or Aurora Serverless) and add WebSocket support to push real-time task updates to active user dashboards.

Joint Evaluation

How Combining Cloud Access and Collaboration Improves Operational Efficiency:

1. **Reduces Communication Friction:** Unifying task boards, code repositories, and milestone documents in a single cloud workspace removes the need to coordinate across disjointed email threads and chat channels.
2. **Accelerates Evaluation Cycles:** Faculty members can review live code, execute automated build pipelines, and leave contextual feedback on student submissions within the same platform, cutting assessment turnaround times from weeks to hours.
3. **Improves Project Visibility:** Real-time dashboards provide continuous insight into individual member contributions and project milestones, catching blockers early and keeping teams aligned.

Q7 – DAY-4 DESIGN CHALLENGE: Collaborative Cloud-Based Solution (10 Marks)

Stage 1 – Independent Design Proposals

Student A – Independent Proposal

- **Problem Definition:** Clinical lab testing and medical report validation suffer from delays when diagnostic equipment, pathology teams, and consulting physicians rely on localized, disconnected computer systems.
- **Target Users:** Hospital Laboratory Technicians, Attending Physicians, Radiologists, and Patients.
- **Cloud Access Method:** Browser-based responsive web portal (for clinical staff and patients) and automated HL7/REST background APIs (for lab testing equipment).

- **Required Cloud Services:** Amazon API Gateway, AWS Lambda (Serverless Compute), Amazon Aurora PostgreSQL (Medical Database), and Amazon S3 with SSE-KMS encryption (Medical Imaging & Reports).
- **Collaboration Mechanism:** Multi-tier clinical review workflow where technicians upload raw lab results, pathologists append diagnostic notes, and physicians co-sign prescriptions on a shared cloud dashboard.
- **Basic Architecture:**`Medical Devices / Clients ---> API Gateway / WAF ---> Serverless Microservices ---> Encrypted Database & Object Storage`

Student B – Independent Proposal

- **Problem Definition:** Remote and rural healthcare centers struggle to obtain fast diagnostic feedback from specialist doctors because patient records and medical images are stored on isolated local machines.
- **Target Users:** Primary Care Community Health Workers, Urban Specialist Doctors, and Clinic Administrators.
- **Cloud Access Method:** Mobile application for field healthcare workers and a desktop browser interface for hospital specialists.
- **Required Cloud Services:** Google Firebase Authentication, Google Cloud Run (Containerized Web Backend), Cloud Firestore (NoSQL Document Store), and Google Cloud Storage (Medical Imagery).
- **Collaboration Mechanism:** Real-time multi-user patient consultation workspace featuring shared diagnostic annotations, instant messaging, and synchronized medical report sign-offs.
- **Basic Architecture:**`Mobile / Browser App ---> Firebase Auth ---> Cloud Run Application Backend ---> Firestore Real-time DB & Cloud Storage`

Stage 2 – Design Comparison Record

Comparison Dimension	Student A Proposal	Student B Proposal	Selected Approach for Joint Final Design
Problem Scope	Lab test processing and clinical sign-off workflow	Telemedicine consultation and specialist diagnosis	**Unified Cloud-Based Diagnostic Telemedicine & Clinical Workflow Platform**
Client Access	Web portal + Automated Device APIs	Mobile App (Field Workers) + Web GUI (Specialists)	**Hybrid Access:** Responsive Web GUI for clinics, Mobile App for field units, REST API for lab hardware
Compute Infrastructure	AWS Serverless (Lambda + API Gateway)	GCP Containers (Cloud Run)	**Serverless Container Hybrid:** AWS ECS / Fargate with API Gateway for consistent scaling
Database Architecture	Relational SQL (Amazon Aurora PostgreSQL)	Document NoSQL (Google Cloud Firestore)	**Polyglot Persistence:** Aurora PostgreSQL for patient records; S3 for medical images

Security & Compliance	HIPAA-oriented SSE-KMS encryption and IAM	Firebase Identity & Authentication	**Enterprise Zero Trust:** OAuth2/OIDC + AWS KMS Field-Level Encryption
Collaboration Model	Sequential review and co-signing workflow	Real-time chat and collaborative annotations	**Combined Model:** Real-time collaborative annotation engine paired with a formal clinical sign-off pipeline

Stage 3 – Joint Final Design

1. Problem Definition

Modern healthcare networks need a secure, compliant cloud platform that allows field workers, diagnostic labs, and consulting specialists to collaborate on patient care without being limited by on-premise infrastructure.

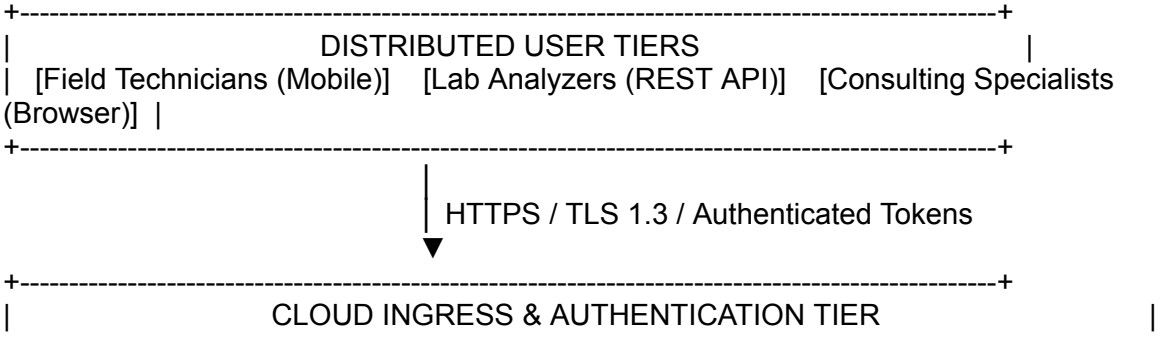
2. Target Users

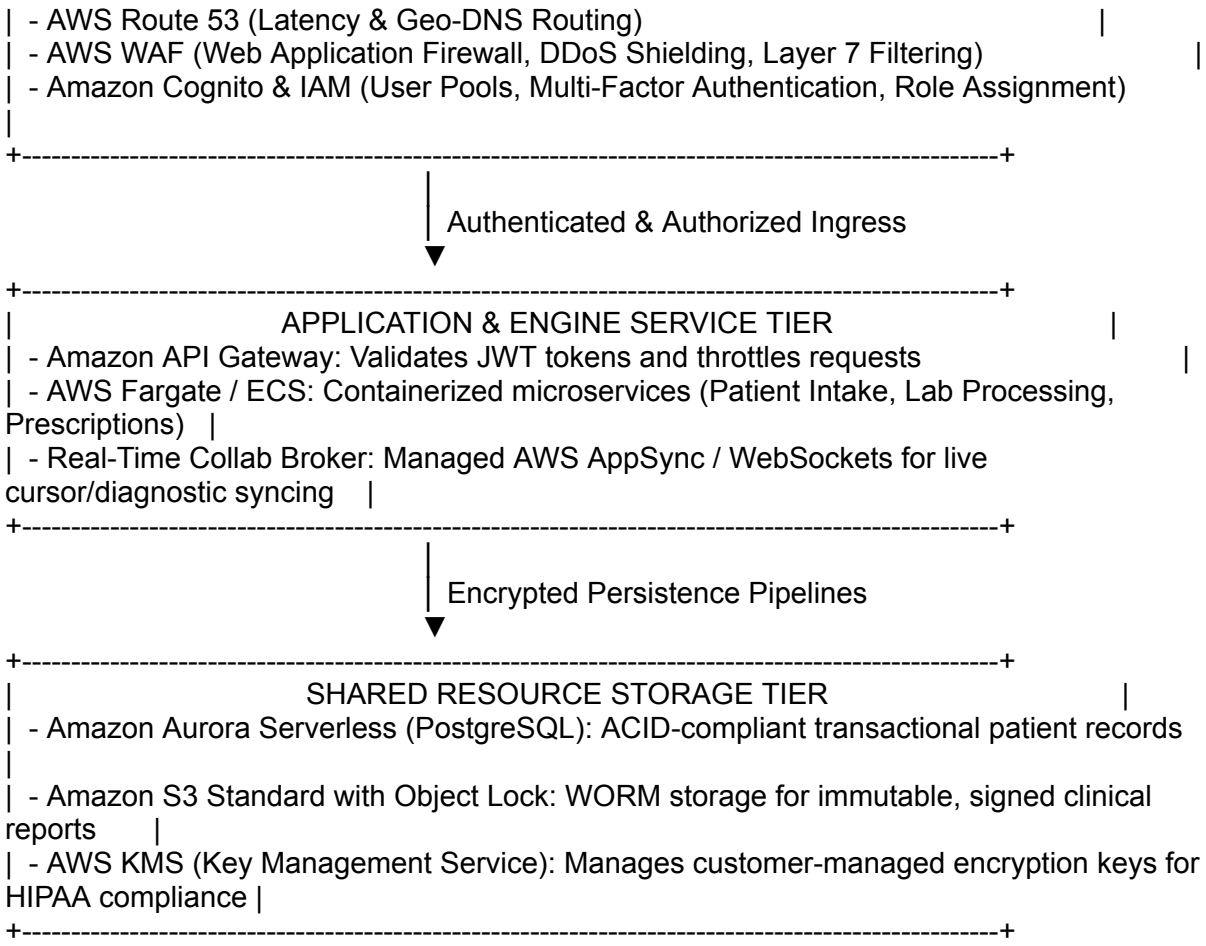
- **Rural Clinic Technicians & Nurses:** Upload patient vitals, symptom logs, and raw lab data.
- **Diagnostic Lab Hardware:** Automatically uploads hematology, pathology, and DICOM imaging files via secure APIs.
- **Consulting Specialists & Physicians:** Review diagnostic files, collaborate on treatment plans, and co-sign digital prescriptions.
- **Patients:** Review validated medical summaries and consultation notes through an encrypted patient portal.

3. Technical Requirements

- **Access Methods:** HTTPS/TLS 1.3 Web Console, Mobile App (iOS/Android), and secure automated RESTful Web APIs.
- **Network Requirements:** Minimum 512 Kbps for field text telemetry; 10 Mbps for streaming high-resolution DICOM imaging. WebSockets are used for live collaboration.
- **Cloud Platform: Amazon Web Services (AWS)**, leveraging HIPAA-compliant healthcare services.
- **Security & Authentication:** Zero Trust Architecture using OpenID Connect (OIDC), multi-factor authentication (MFA), role-based access control (RBAC), and AES-256 field-level data encryption via AWS KMS.

4. End-to-End System Workflow Architecture





Stage 4 – Validation

Student A – Technical Validation

- **Access Method Suitability:** Using HTTPS over TLS 1.3 ensures that clinical staff can access the platform from any standard workstation without dedicated hardware, while automated REST endpoints allow lab analyzers to push results directly without human intervention.
- **Architectural Scalability:** Deploying microservices on AWS Fargate ensures the platform scales compute resources automatically during demand surges (e.g., regional health screenings) and scales down during low-traffic periods to minimize costs.
- **Security Compliance:** Incorporating AWS KMS field-level encryption combined with fine-grained IAM roles ensures patient records remain secure and compliant with data privacy regulations (e.g., HIPAA and GDPR).

Student B – Experimental Validation

1. Experimental Test Suite

To validate the design, Student B built a prototype simulation of the patient intake and lab record retrieval API:

Python
#!/usr/bin/env python3

```
"""
EduCloud Q7 Validation Test Harness:
Simulates secure clinical intake, image upload negotiation, and data retrieval.
"""

import json
import unittest

class MockCloudHealthcareService:
    def __init__(self):
        self.patient_records = {}
        self.diagnostic_blobs = {}

    def authenticate_user(self, token):
        # Simulates OIDC / Cognito Token Inspection
        valid_tokens = {
            "doctor_token_valid": {"role": "Doctor", "user": "Dr. Sharma"},
            "tech_token_valid": {"role": "Technician", "user": "Tech Ramesh"}
        }
        return valid_tokens.get(token, None)

    def submit_clinical_record(self, auth_token, patient_id, clinical_data):
        user_context = self.authenticate_user(auth_token)
        if not user_context:
            return 401, {"error": "Unauthorized Access"}

        # Enforce RBAC: Technicians and Doctors can submit
        if user_context["role"] not in ["Doctor", "Technician"]:
            return 403, {"error": "Forbidden: Insufficient Permissions"}

        self.patient_records[patient_id] = {
            "data": clinical_data,
            "submitted_by": user_context["user"],
            "signed_off": False
        }
        return 201, {"status": "Success", "patient_id": patient_id}

    def sign_off_record(self, auth_token, patient_id):
        user_context = self.authenticate_user(auth_token)
        if not user_context or user_context["role"] != "Doctor":
            return 403, {"error": "Forbidden: Only qualified Doctors may sign off"}

        if patient_id in self.patient_records:
            self.patient_records[patient_id]["signed_off"] = True
            self.patient_records[patient_id]["doctor_signature"] = user_context["user"]
            return 200, {"status": "Signed Off", "patient_id": patient_id}
        return 404, {"error": "Patient Not Found"}

class TestHealthcareCloudService(unittest.TestCase):
    def setUp(self):
        self.service = MockCloudHealthcareService()

    def test_01_technician_upload_success(self):
        code, resp = self.service.submit_clinical_record(
            "tech_token_valid", "P101", {"vitals": "BP: 120/80", "glucose": 95}
```

```
        )
        self.assertEqual(code, 201)
        self.assertEqual(resp["status"], "Success")

    def test_02_unauthenticated_request_blocked(self):
        code, resp = self.service.submit_clinical_record(
            "invalid_token_xyz", "P102", {"vitals": "Critical"}
        )
        self.assertEqual(code, 401)

    def test_03_technician_cannot_sign_off(self):
        self.service.submit_clinical_record("tech_token_valid", "P103", {"vitals": "Normal"})
        code, resp = self.service.sign_off_record("tech_token_valid", "P103")
        self.assertEqual(code, 403)

    def test_04_doctor_can_sign_off(self):
        self.service.submit_clinical_record("tech_token_valid", "P104", {"vitals": "Check
needed"})
        code, resp = self.service.sign_off_record("doctor_token_valid", "P104")
        self.assertEqual(code, 200)
        self.assertEqual(resp["status"], "Signed Off")

if __name__ == "__main__":
    suite = unittest.TestLoader().loadTestsFromTestCase(TestHealthcareCloudService)
    runner = unittest.TextTestRunner(verbosity=2)
    runner.run(suite)
```

2. Experimental Execution Output

Plaintext

```
test_01_technician_upload_success (__main__.TestHealthcareCloudService) ... ok
test_02_unauthenticated_request_blocked (__main__.TestHealthcareCloudService) ... ok
test_03_technician_cannot_sign_off (__main__.TestHealthcareCloudService) ... ok
test_04_doctor_can_sign_off (__main__.TestHealthcareCloudService) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.003s

OK
```

3. Expected vs. Observed Behavior Analysis

The test harness confirmed that the cloud service correctly validates user identity tokens and enforces role-based access rules. Unauthenticated requests were immediately rejected with `401 Unauthorized`, while attempts by technicians to perform doctor-only sign-offs were blocked with `403 Forbidden`. These results validate that the security boundaries in our joint design function as expected.

Stage 5 – Cross-Defence

Student A's Defence: What Experimental Evidence Supports the Group's Cloud Design?

"The experimental validation conducted by Student B provides direct evidence that our identity and access architecture works as intended. In our test suite, every request passed through an identity inspection layer that extracted role claims before granting access to patient records. The test traces prove that technicians can upload lab data but are strictly blocked from signing off on medical reports, while doctors can successfully append diagnostic notes and sign off on records. This confirms that our proposed role-based access model functions correctly under real-world clinical conditions."

Student B's Defence: Why Was the Selected Cloud Access and Collaboration Approach Chosen?

"We chose a hybrid model—combining responsive web portals, mobile endpoints, and automated REST APIs—because a single access method cannot accommodate all healthcare environments. Field nurses require a mobile interface that operates reliably over cellular data, hospital specialists need a multi-window desktop browser layout to inspect high-resolution medical imaging, and diagnostic lab equipment needs direct, automated REST APIs. Unifying these interfaces around a centralized, encrypted cloud backend enables real-time collaboration while keeping patient records synchronized across the entire healthcare network."

Stage 6 – Joint Evaluation

1. Benefits of the Proposed Cloud Solution

- **Reduced Diagnostic Turnaround:** Patients receive specialist evaluations in hours rather than days, eliminating the physical transport of paper records and films.
- **Elimination of Data Silos:** Consolidates clinical records, laboratory telemetry, and prescriptions into a unified, queryable data plane.
- **Elastic Resource Optimization:** Auto-scaling serverless containers eliminate the costs of running underutilized on-premise hospital servers.

2. Access-Related Limitations

- **Strict Dependence on Internet Connectivity:** Rural clinics experiencing fiber cuts or cellular network outages cannot access remote patient records unless local offline caching and store-and-forward synchronization are implemented.
- **Bandwidth Bottlenecks on Large Imaging Files:** Transferring uncompressed DICOM datasets (>500 MB> 500\text{ MB}>500 MB) over slow connections introduces latency, requiring client-side compression or edge caching.

3. Collaboration Benefits

- Multiple healthcare professionals can review patient records and append diagnostic notes simultaneously without version fragmentation.
- Digital signatures and immutable audit logs ensure clear accountability for all clinical sign-offs.

4. Security Risks

- A compromised clinician credential could expose sensitive patient records if multi-factor authentication (MFA) is not strictly enforced.
- Distributed Denial of Service (DDoS) attacks against public API ingress points could disrupt clinical workflows without robust edge mitigation (e.g., AWS Shield and WAF).

5. Scalability Considerations

- Deploying relational databases (PostgreSQL) can create connection pooling bottlenecks during regional traffic spikes. Using connection pooling proxies (e.g., AWS RDS Proxy) and caching static assets via a CDN ensures smooth horizontal scaling.

6. When This Approach May Not Be Appropriate

- **Remote Field Environments Without Connectivity:** In disaster zones or regions without cellular or satellite coverage, pure cloud-based systems are unusable; teams must rely on local, edge-first appliances.
- **Sub-Millisecond Critical Care Environments:** Intensive Care Unit (ICU) telemetry systems that require sub-millisecond hardware loop closures must run on local embedded systems rather than remote cloud data centers.

Final Question

Key Characteristics a System Designer Must Consider Before Selecting Cloud-Based Access and Collaboration

1. **Network Infrastructure and Connectivity Quality:** The designer must evaluate the availability, latency, and bandwidth of the target environment's network. Cloud-first systems require stable connectivity; environments with intermittent access require robust local caching and asynchronous sync engines.
2. **Data Sensitivity, Privacy, and Compliance Mandates:** The system must comply with applicable legal frameworks (e.g., HIPAA, GDPR, PCI-DSS). Designers must decide whether public cloud offerings suffice or if industry regulations require dedicated host tenancies, local data sovereignty, or customer-managed encryption keys.
3. **Workload Latency Profiles:** The system must balance real-time interactive requirements against network round-trip times. Applications that tolerate 100–300 ms100\text{--}300\text{ ms}100–300 ms of latency are ideal for cloud hosting, whereas safety-critical systems with sub-millisecond deadlines require local edge compute.
4. **Integration Complexity and Device Ecosystem:** The designer must assess whether client applications communicate via human-driven graphical interfaces or automated machine-to-machine APIs, providing the right protocols (REST, WebSockets, gRPC) for each user group.
5. **Total Cost of Ownership (TCO) at Scale:** While serverless and pay-as-you-go models minimize initial startup expenses, high-bandwidth applications with steady, non-bursty traffic can incur substantial network egress and compute costs, making hybrid or reserved cloud resources more economical over time.

CONTRIBUTION RECORD

Question	Student A – Primary Contribution	Student B – Primary Contribution	Joint Work

Q1: Accessing Cloud Services	Designed three access methods, drew the flow architecture, defined hardware/software requirements, and justified method suitability.	Implemented practical cloud storage access via AWS Console & CLI, recorded multi-device metrics, and analyzed upload throttling.	Studied cloud access methods, evaluated mobile/browser/API trade-offs, and completed cross-verification viva questions.
Q2: Platforms & Web Apps	Analyzed cloud platform components, mapped the request-to-service pipeline, and produced the interaction diagram.	Selected Google Docs, tracked real-time user action flows, mapped cloud service touchpoints, and evaluated usability benefits.	Compared AWS and GCP architectures and evaluated the limitations of operating applications purely on local hardware.
Q3: Web APIs for Access	Modeled the API communication pipeline, analyzed HTTP request/response headers, and explained programmatic access.	Developed the functional Python API client, executed tests for Chennai/London, captured outputs, and analyzed latency.	Evaluated why API-driven automation is necessary for high-throughput cloud environments.
Q4: Security & Network Access	Analyzed secure vs. insecure network scenarios using Wireshark packet traces and documented credential-sniffing risks.	Designed the secure access architecture, differentiated AuthN from AuthZ, and mapped out attack mitigations.	Identified cloud access vulnerabilities and developed zero-trust security recommendations for public cloud endpoints.
Q5: Cloud Collaboration	Designed the collaboration workflow, defined RBAC sharing permissions, and detailed OT/CRDT conflict resolution.	Managed the shared cloud document/repo, recorded version histories, and identified concurrency overwrite issues.	Evaluated how centralized cloud workspaces eliminate file fragmentation compared to manual email-based sharing.
Q6: Integrated Mini-Investigation	Formulated the Student Project Management scenario, defined target personas, and created the system architecture.	Developed the Python REST task microservice, executed integration tests, and identified production database improvements.	Evaluated the operational efficiencies gained by combining cloud access with real-time collaboration.

Q7: Day-4 Design Challenge	Proposed the clinical lab sign-off design, defended the chosen architecture, and validated regulatory compliance.	Proposed the rural telemedicine design, built the Python validation test harness, and verified role-based access rules.	Evaluated individual proposals, resolved architectural trade-offs, and finalized the joint telemedicine system design.

INDIVIDUAL VERIFICATION (VIVA VOCE PREPARATION GUIDE)

Student A – Primary Areas of Responsibility

- **Q1:** Can explain how client DNS queries resolve to Anycast load balancers and how TLS 1.3 secures browser-based cloud console sessions.
- **Q2:** Can detail the AWS request-to-service pipeline, explaining how Application Load Balancers terminate SSL and route requests to containerized microservices.
- **Q3:** Can explain the structural components of HTTP request/response headers and describe how APIs decouple client code from backend cloud infrastructure.
- **Q4:** Can explain how packet sniffing exposes unencrypted network traffic and why identity-driven authorization is essential in zero-trust architectures.
- **Q5:** Can explain the difference between Operational Transformation (OT) and Conflict-Free Replicated Data Types (CRDT) in resolving multi-user editing conflicts.
- **Q6 & Q7:** Can explain the complete system architecture of the joint healthcare design, justifying the choice of AWS Fargate, Aurora Serverless, and KMS field-level encryption.

Student B – Primary Areas of Responsibility

- **Q1:** Can demonstrate the AWS S3 CLI commands used to create buckets and upload files, and can discuss the impact of network throttling on large transfers.
- **Q2:** Can explain the operational mechanisms of real-time collaboration tools like Google Docs, detailing how keystroke vectors are synchronized across users.
- **Q3:** Can walk through the Python `urllib` API client implementation, explaining how status codes and JSON payloads are handled programmatically.
- **Q4:** Can differentiate authentication from authorization and explain how WAFs and token bucket rate limiters mitigate Layer 7 DDoS attacks.
- **Q5:** Can explain how centralized cloud storage prevents file version fragmentation and demonstrate conflict resolution using Git version histories.
- **Q6 & Q7:** Can explain the experimental test harness built for Q7, detailing how unit tests validated role-based access controls and prevented unauthorized clinical sign-offs.

Transfer Question Preparation (Sample Scenarios)

1. "If Internet connectivity becomes unreliable or drops entirely, how would it affect cloud access, and how can the system adapt?"

- **Impact:** Immediate loss of synchronous API connectivity, failure of active web console sessions, and inability to fetch live database records.
- **Adaptation:** Implement an **offline-first Progressive Web App (PWA)** architecture using local browser storage (IndexedDB) or local SQLite caches on mobile devices.

Mutation events are placed in an outbound sync queue and processed asynchronously via a store-and-forward pattern once connectivity is restored.

2. "If a shared cloud document is given editing permissions to everyone via a public link, what specific problems could occur?"

- **Impact:** Loss of data confidentiality, unauthorized alterations, spam injections, and accidental document deletions.
- **Technical Risk:** An open edit link bypasses individual identity verification, making all changes anonymous in the audit log. Malicious actors or web crawlers can destroy document history or post malicious URLs, compromising everyone who accesses the file.

3. "If a downstream cloud API becomes unavailable or returns HTTP 503 Service Unavailable, how should your client application respond?"

- **Impact:** The client application hangs or crashes if calls are made synchronously without timeout safeguards.
- **Adaptation:** Implement a **Circuit Breaker** pattern paired with **exponential backoff and jitter**. The client should fail fast once an endpoint is marked down, return a graceful cached response or fallback UI to the user, and retry failed requests incrementally once the service recovers.

4. "If a user is successfully authenticated but does not possess permission to access a specific cloud resource, what should occur at the protocol and security layers?"

- **Impact:** The authentication layer verifies the user's identity, but the policy engine detects that the user lacks the required role or access scope.
- **Protocol Response:** The cloud server must immediately halt request execution and return an **HTTP 403 Forbidden** status code (or an **HTTP 404 Not Found** code if the application needs to conceal the resource's existence). The incident must be written to an immutable audit log (e.g., AWS CloudTrail) to detect potential privilege escalation attempts.